Solving Dynamic Programming Problems using Code Generation

By Michael Lin (CS), Matthew Samuels (CS), Charles Anton Sibal (CS)
Advisor: George Heineman (CS)

Background

- Investigated Dynamic Programming as an algorithmic paradigm
  - Bottom-up
  - Top-down, with naïve vs memoized solutions
- Domain-Specific Languages
  - Dynprog (R)
- DP Problem Examples in Java
- Abstract Syntax Trees
- CoGen

Design

- Developed `Dynamic-Programming-MQP` Git repository
  - Categorized DP problems
  - Implemented in Java (with some Dynprog)
- Developed `expression-problem/tree/IL-DP` Git repository
  - Hand-coded CoGen solutions of DP problems
  - Identified and factored out helper methods
  - Created high-level metamodel of DP problems
  - Developed a generator to consume metamodels and utilize CoGen to generate solutions

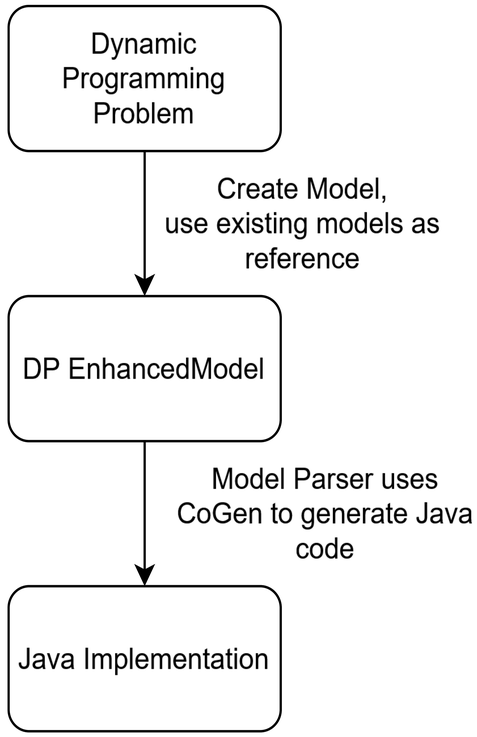
Future Work

- Research more complex DP problems
  - NLP; images; graphs
- Explore other languages
  - Haskell; python
- Subproblem traversal order
- Family re-classification

Acknowledgments

The MQP team thanks Professor George Heineman for his support, guidance, and patience throughout the project's duration.

Enhanced Model Pipeline

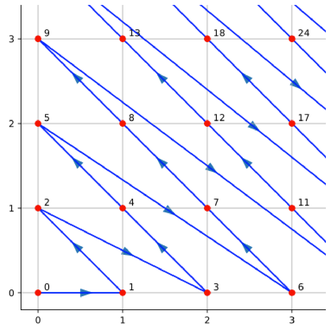


Conclusions

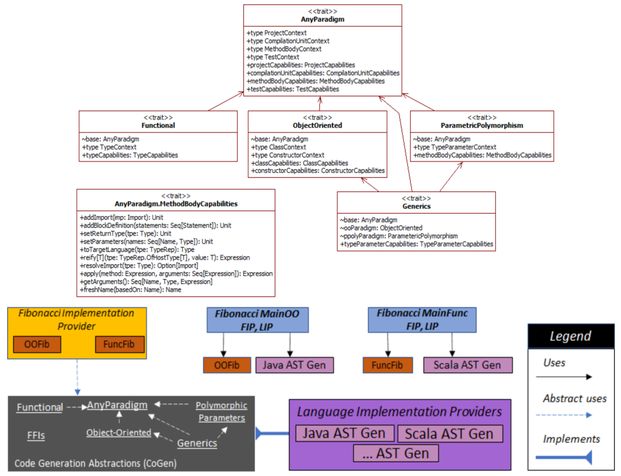
- Developed repositories:
  - Successfully implemented 21 EnhancedModel DP instances
  - Buildable and testable code with Gradle and SBT
  - Utilized to generate solutions regardless of programming language
- Gained deeper understanding of Dynamic Programming
  - Problems are more natural to categorize on the nature of their subproblem structure
  - Language-agnostic view of DP

Cantor Pairing

For top-down implementations, we needed to encode multiple integers into one hashable value. We used a formula invented by Cantor that uniquely maps 2 integers into 1 integer. This can be chained any number of times to scale for more inputs.



CoGen Framework



Path retrieval for Longest Common Subsequence

		0	1	2	3	4
			B	D	C	B
0		0	0	0	0	0
1	B	0	1	1	1	1
2	A	0	1	1	1	1
3	C	0	1	1	2	2
4	D	0	1	2	2	2
5	B	0	1	2	2	3